

Memoria Práctica I y II de Física II

Hugo García Poza

Índice

Metodología	1
Estructura del robot.yaml	2
Estructura del archivo class_robot_structure.py	2
¿Por qué se usan dos clases: Robot y Link?	2
Flujo de uso básico	2
Verificación del estado del robot	3
Archivo estructura	3
Práctica 1	4
7.4. Programación del Niryo One con Python.	4
7.4.1. Movimiento P1-P2 y 7.4.2. Mover un objeto ligero.	4
7.5. Resolución del problema cinemático directo para el Niryo One.	5
Código para mover el robot en un ciclo de posiciones:	6
Práctica 2	6
Punto 7.1: Matriz Jacobiana: ejercicios.py -> Opcion 10	7
Punto 7.2 : Encontrar singularidades.	7
Punto 7.3: Comparación de configuraciones y elipsoides de manipulabilidad.	9
Config 0 Config 1 Config 2 Singularidad	9
Punto 7.4. Cinemática Inversa.	10

Metodología

A lo largo del curso he ido desarrollando los códigos necesarios para estas prácticas y los seminarios, esto lo he hecho principalmente adaptando los códigos dados, generalizando y pasando a Python 3.12, ahora bien el código de seminarios y prácticas está interconectado. Para validar tanto los códigos como archivos de estructuras use el código del profesor para el Niryo One. Los problemas de encontrar una solución a un sistema los he resuelto de forma numérica a excepción de la búsqueda de singularidades que usa un método analítico con cálculo simbólico.

Mencionar que las imágenes del código/terminal pueden diferir sobre el código final puesto, ya que se ha corregido y normalizado ciertas partes del código. Para verlo completo y junto su documentación puedes acceder a github.com/Huxsby/physics-II. Para probar el código es necesario tener todas las librerías usadas instaladas y abrir los programas desde una misma carpeta ya que se agrupó el tipo de cálculo por archivo y en ejercicios.py los problemas resueltos.

Estructura del robot.yaml

Estructura del archivo `class_robot_structure.py`

Este módulo define cómo representar la estructura de un robot manipulador en Python. Está compuesto principalmente por dos clases: `Robot` y `Link`. También contiene una función `cargar_robot_desde_yaml()` que permite construir un objeto `Robot` completo a partir de un archivo de configuración YAML, como el que usamos para definir los parámetros físicos de nuestro robot real (`robot.yaml`). Paralelamente también hay varias funciones 'límite' que trabajan con los límites de cada articulación y se aseguran que trabajemos dentro de los mismos, generar coordenadas aleatorias, comprobar configuraciones concretas, filtrar un conjunto de estas, etc. Sobre estos límites destacar que se implementó la opción de trabajar con un margen predefinido de 5° (a rad) en `MARGEN_LIMITES_THETAS`. Aunque por claridad de los datos no se usó a lo largo de la práctica, ni estará activo en el código final.

¿Por qué se usan dos clases: `Robot` y `Link`?

Por su **mayor generalización**. Separar la clase `Link` (que representa cada eslabón del robot, con su tipo de articulación, eje, coordenadas, etc.) de la clase `Robot` (que simplemente agrupa todos los eslabones y maneja la lógica general del robot) permite una estructura más modular, clara y reutilizable.

- `Link`: contiene toda la información específica de un único eslabón del robot.
- `Robot`: se encarga de organizar todos los `Link` y calcular propiedades más globales, como los **ejes helicoidales**, que son fundamentales para la cinemática directa.

Flujo de uso básico

1. **Cargar el robot:**
Se llama a la función `cargar_robot_desde_yaml()` indicando el archivo `.yaml`. Esta función construye un objeto `Robot`, lee todos los eslabones definidos, los convierte en objetos `Link` y los añade al robot.

```
robot = cargar_robot_desde_yaml("robot.yaml")
```
2. **Usar el robot:**
Una vez cargado, se pueden consultar distintas propiedades, como `robot.links` (lista de eslabones) o `robot.ejes_helicoidales`, que se calculan automáticamente al cargar el robot.

```
print(robot.ejes_helicoidales)
```
3. **Inspección individual:**
Para verificar la dirección y sentido de cada articulación, se puede llamar al método `.obtener_eje_de_giro()` de cada `Link`. Esto nos ayudó a detectar posibles errores como ejes invertidos por conexiones mal hechas en el robot real.
4. **Validación de ángulos:**
Para verificar la dirección y sentido de cada articulación, se puede llamar a la función `limits(robot)` para verificar que un array de thetas cumple con los límites establecidos en las articulaciones, `get_limits_positive(robot)` o `get_limits_negative(robot)` para obtener dichos valores límite. Todo esto con una variable de margen (para evitar colisiones o torcer cables) definida al principio de la clase.

Verificación del estado del robot

Usando la opción 8 de ejercicios.py podemos obtener la información relevante de la estructura del robot y comprobar aumentando/disminuyendo los ángulos en cada articulación del robot real podemos verificar si cumple correctamente con la estructura previamente definida. Y cambiar en el .yaml, por ejemplo los ejes de rotación (los cables de algún motor pudieron haber sido invertidos en alguna reparación), etc. Para esto se llama al método .obtener_eje_de_giro() en cada link (eslabón).

```
0. Salir.

Seleccione una opción: 8
Lectura de archivo YAML (robot.yaml)
Robot 'niryo_one' creado.
Tiempo de ejecución de get_ejes_helicoidales: 0.0002 segundos

Robot 'niryo_one' con 6 eslabones.
El Eslabón 'Base' (revolute), coordenadas: [0. 0. 0.103], eje: [0 0 1], longitud: 0.103
El Eslabón 'Hombro' (revolute), coordenadas: [0. 0. 0.08], eje: [0 -1 0], longitud: 0.08
El Eslabón 'Brazo' (revolute), coordenadas: [0. 0. 0.21], eje: [0 -1 0], longitud: 0.21
El Eslabón 'Codo' (revolute), coordenadas: [0.0415 0. 0.03 ], eje: [1 0 0], longitud: 0.03
El Eslabón 'Antebrazo' (revolute), coordenadas: [0.18 0. 0. ], eje: [0 -1 0], longitud: 0.0415
El Eslabón 'Muñeca' (revolute), coordenadas: [ 0.0237 0. -0.0055], eje: [1 0 0], longitud: 0.18

Ejes helicoidales del robot: [array([ 0., 0., 1., -0., -0., -0.]), array([ 0. , -1. , 0. , 0.183, -0. , -0. ]), array([ 0. , -1. , 0. , 0.39
3, -0. , -0. ]), array([ 1. , 0. , 0. , -0. , -0. , 0.423, -0. ]), array([ 0. , -1. , 0. , 0.423, -0. , -0.2215]), array([ 1. , 0.
, 0. , -0. , 0.4175, -0. ])]

Obtener eje de giro
Eslabón Base: Eje de giro: +Z ( sentido positivo == horario u )
Eslabón Hombro: Eje de giro: -Y ( sentido negativo == antihorario u )
Eslabón Brazo: Eje de giro: -Y ( sentido negativo == antihorario u )
Eslabón Codo: Eje de giro: +X ( sentido positivo == horario u )
Eslabón Antebrazo: Eje de giro: -Y ( sentido negativo == antihorario u )
Eslabón Muñeca: Eje de giro: +X ( sentido positivo == horario u )

Presione Enter para continuar...[]
```

Una vez verificado esto es seguro trabajar con el robot. Esto es algo que es mejor hacer antes de empezar a trabajar en las prácticas puesto de no ser correcto, los cálculos tampoco lo serán, algo que Sacarino nos hizo aprender por las malas, al tener más de un eje al revés. La solución a esto es o bien modificar el archivo .yaml, o tomar otro brazo y repetir el proceso. Sacarino estaba teniendo problemas de estabilidad ese día (y el resto también) así que al final de las prácticas estábamos usando a Mortadelo.

(Más adelante se expondrán las salidas del resto del código que son cálculos y estrategias para resolver los diferentes problemas que se nos fueron planteando, para ver como son por dentro puedes consultar su repositorio de github compartido al inicio de la memoria)

Archivo estructura

```
robot:
  name: niryo_one
  links:
    # Base (J1)
    - id: Base
      length: 0.103
      type: revolute
      link_orientation: [0, 0, 1]
      joint_coors: [0, 0, 0.103]
      joint_axis: [0, 0, 1]

    # Hombro (J2)
    - id: Hombro
      length: 0.088
      type: revolute
      link_orientation: [0, 1, 0]
      joint_coors: [0, 0, 0.088]
      joint_axis: [0, -1, 0]

    # Brazo (J3)
    - id: Brazo
      length: 0.210
      type: revolute
      link_orientation: [0, 1, 0]
      joint_coors: [0, 0, 0.210]
      joint_axis: [0, -1, 0]

    # Codo (J4) - Corrección de eje
    - id: Codo
      length: 0.030
      type: revolute
      link_orientation: [0, 1, 0]
      joint_coors: [0.0415, 0, 0.030]
      joint_axis: [1, 0, 0] # Eje X

    # Antebrazo (J5) - Corrección de eje
    - id: Antebrazo
      length: 0.0415
      type: revolute
      link_orientation: [1, 0, 0]
      joint_coors: [0.180, 0, 0]
      joint_axis: [0, -1, 0] # Eje -Y

    # Muñeca (J6) - Corrección de eje y coordenadas
    - id: Muñeca
      length: 0.180
      type: revolute
      link_orientation: [1, 0, 0]
      joint_coors: [0.0237, 0, -0.0055] # Posición corregida
      joint_axis: [1, 0, 0] # Eje X
```

Práctica 1

A lo largo de esta práctica aprendemos a cómo manejar el el Niryo One tanto con su interfaz gráfica como API de python y las precauciones de cómo manejarlo, para evitar que el robot se caiga, retuerza los cables o forzar los motores. Y los preparativos para empezar a usarlo, moverlo para verificar sus ejes de giro, autocalibrar, el modo aprendizaje, etc. Así como entender estructura y articulaciones del mismo, algo fundamental para poder hacer un robot.yaml preciso y correcto.

En los ejercicios de esta práctica aprendemos a cómo moverlo y manejarlo en modo aprendizaje y realizar una secuencia de movimientos basándonos en los ejemplos dados. Asíque mejor centrémonos en los últimos puntos que combina todo la dado en los anteriores:

7.4. Programación del Niryo One con Python.

En este punto modificamos el programa del punto 6.4 para hacer que se mueva entre P1 y P2, 4 veces. Y por otra parte hacemos que agarre una goma para probar la pinza.

[VID-20250410-WA0001.mp4](#) (Movimiento P1-P2)

[VID-20250410-WA0004.mp4](#) (Mover una goma)

7.4.1. Movimiento P1-P2 y 7.4.2. Mover un objeto ligero.

```
#!/usr/bin/env python
from niryo_one_python_api.niryo_one_api import *
import rospy
import time
rospy.init_node('niryo_one_example_python_api')
n = NiryoOne()
n.calibrate_auto()
n.activate_learning_mode(False)
# desactiva el modo de aprendizaje
for i in range(0, 5):
    n.move_pose(-0.03, -0.156, 0.48, -0.58, -0.58, -0.145) # mueve el robot a P1
    time.sleep(3)
    pose_actuel_1 = n.get_arm_pose()
    print pose_actuel_1
    n.move_pose(-0.136, -0.133, 0.255, -0.081, 0.744, -2.535)
# mueve el robot a P2
    time.sleep(3)
    pose_actuel_2 = n.get_arm_pose()
    print pose_actuel_2
n.activate_learning_mode(True)
```

```
#!/usr/bin/env python

"""
Programa para controlar la pinza del robot
"""

from niryo_one_python_api.niryo_one_api import *
import rospy
import time
rospy.init_node('niryo_one_example_python_api')

# Inicializamos el brazo
brazo = NiryoOne()

#Iniciamos el robot y hacemos la calibracion automatica
brazo = NiryoOne()
brazo.calibrate_auto()

# Intentamos mover la pinza
try:
    # Le indicamos al robot que cambie a la pinza normal
    brazo.change_tool(TOOL_GRIPPER_1_ID)

    #le damos un tiempo
    time.sleep(3)

    #mueve el brazo a la posicion P1
    brazo.move_pose(-0.03, -0.156, 0.48, -0.58, -0.58, -0.145)
    time.sleep(3)
    pose_actuel_1 = brazo.get_arm_pose()
    print pose_actuel_1

    # Le indicamos al robot que abra la pinza y espero 2 segundos
    brazo.open_gripper(TOOL_GRIPPER_1_ID, 500)
    brazo.wait(2)

except NiryoOneException as e:
    # En caso de error imprime el fallo
    print e

#Activamos el modo aprendizaje
brazo.activate_learning_mode(True)
```

7.5. Resolución del problema cinemático directo para el Niryo One.

El objetivo de este punto fue probar nuestro cálculo de la transformada homogénea y comprobar que fuera correcto moviendo el robot. Para esto hicimos varias pruebas introduciendo las coordenadas en [ejercicios.py](#) -> Opción 9. Empezamos validamos el código, con la matriz M (Transformación homogénea de la base al efector final) y seguimos con las siguientes coordenadas fijándonos en la interfaz del propio niryo para verificar los ángulos de Euler.

Versión usada (Usamos el código de cinemática directa para ir validando):

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  COMENTARIOS
Matriz M (posición cero):
M =
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Valores de las articulaciones: [0, 0, 0, 0, 0, 0]

Matriz de transformación homogénea T:
T =
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Coordenadas (x,y,z) del TCP: [0.2452 0.   0.4175]
Los angulos de Euler (Roll Pitch Yaw) son: [ 0. -0.  0.]
Presione Enter para continuar...

sh-5.2$ /usr/bin/python /home/heaven/regit/physics-II/Practica_1
/problema_cinemático_directo.py
Coordenadas en radianes [0. 0. 0. 0. 0.]

Matriz M:
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Matriz de transformación homogénea:
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Coordenadas (x,y,z) del TCP: [0.245 0.   0.418]

Angulos de Euler en radianes [ 0. -0.  0.]

Angulos de Euler en grados:
```

Versión actualizada del código (más fácil de usar):

```
Seleccione una opción: 9
Calcular la matriz de transformación homogénea del robot.
Robot 'brazo_dron' creado.
Tiempo de ejecución de get_ejes_helicoidales: 0.0000 segundos

Ejes helicoidales del robot:
S1: [ 0.0000, 0.0000, 1.0000, -0.0000, -0.0000, -0.0000]
S2: [ 0.0000, -1.0000, 0.0000, 0.1830, -0.0000, -0.0000]
S3: [ 0.0000, -1.0000, 0.0000, 0.3930, -0.0000, -0.0000]
S4: [ 1.0000, 0.0000, 0.0000, -0.0000, 0.4230, -0.0000]
S5: [ 0.0000, -1.0000, 0.0000, 0.4230, -0.0000, -0.2115]
S6: [ 0.0000, 0.0000, 0.0000, 1.0000, 0.0000, 0.0000]
S7: [ 1.0000, 0.0000, 0.0000, -0.0000, 0.4175, -0.0000]

Matriz M (posición cero):
M =
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Límites de las articulaciones (theta):
{'joint_1': (-3.054, 3.054), 'joint_2': (-1.571, 0.6405), 'joint_3': (-1.396, 1.571), 'joint_4': (-3.054, 3.054), 'joint_5': (-1.745, 1.745), 'joint_6': (0.0, 0.15), 'joint_7': (-2.574, 2.574)}
Ingrese los valores de las articulaciones (separados por comas o espacios):
```

Resultado generalizado para cualquier T:

```
Seleccione una opción: 9
Calcular la matriz de transformación homogénea del robot.
Ejes helicoidales del robot: [array([ 0.,  0.,  1., -0., -0., -0.
.]), array([ 0., -1.,  0.,  0.183, -0., -0. ]), array([ 0., -1.,  0.,  0.393, -0., -0. ]), array([ 1.,  0.,  0., -0.,  0.423, -0. ]), array([ 0., -1.,  0.,  0.423, -0., -0.2215]), array([ 1.,  0.,  0., -0.,  0.4175, -0. ])]

Matriz M (posición cero):

M =
[[1.  0.  0.  0.245]
 [0.  1.  0.  0.   ]
 [0.  0.  1.  0.418]
 [0.  0.  0.  1.   ]]

Valores de las articulaciones: [0.7, 0.3, 0.3, 0.4, 0.5, 0.8]

Matriz de transformación homogénea T:

T =
[[ 0.484 -0.84  0.246  0.093]
 [ 0.163 -0.189 -0.968  0.075]
 [ 0.86  0.508  0.046  0.552]
 [ 0.  0.  0.  1.   ]]

Coordenadas (x,y,z) del TCP: [0.09323443 0.07520264 0.55165007]
Los angulos de Euler (Roll Pitch Yaw) son: [ 1.48126585 -1.03521524  0.32547741]
```

Código para mover el robot en un ciclo de posiciones:

```
#!/usr/bin/env python
from niryo_one_python_api.niryo_one_api import *
import rospy
import time
rospy.init_node('niryo_one_example_python_api')
n = NiryoOne()
n.calibrate_auto()
n.activate_learning_mode(False)
# desactiva el modo de aprendizaje
for i in range(0, 16):
    n.move_pose(-0.03, -0.156, 0.48, -0.58, -0.58, -0.145) # mueve el robot a P1
    time.sleep(3)
    pose_actuel_1 = n.get_arm_pose()
    print pose_actuel_1
    n.move_pose(-0.136, -0.133, 0.255, -0.081, 0.744, -2.535)
# mueve el robot a P2
    time.sleep(3)
    pose_actuel_2 = n.get_arm_pose()
    print pose_actuel_2
    n.move_pose(0.223, 0.099, 0.237, 0.153, 0.492, 0.375) # mueve el robot a P3
    time.sleep(3)
    pose_actuel_3 = n.get_arm_pose()
    print pose_actuel_3
n.activate_learning_mode(True)
```

Al final de este punto pudimos validar nuestro código de forma satisfactoria, una vez que nos dimos cuenta de que el problema era Sacarino y no el desarrollador. Así que una vez probado con un niryo con los ejes sin invertir pudimos verificar que las coordenadas coincidan de manera bastante precisa, como esto coincidió con el fin de la sesión no pudimos guardar una captura final.

Práctica 2

En esta sesión nos centramos en desarrollar el código para encontrar las singularidades, graficar los elipsoides de anulabilidad para ciertas configuraciones, y finalmente generalizar el problema cinemático inverso, ver [problema_cinemático_inverso_gen.py](#). Los puntos aquí desarrollados se pueden encontrar en ejercicios.py -> Opciones 10 - 13 (Últimos bloques). Las salidas actuales no coinciden con las del día de la práctica ya que he trabajado en la legibilidad de las mismas.

Destacar de qué se necesita de cierta potencia para encontrar las singularidades y que buscar todas las singularidades posibles no sería viable dado a intensidad de cálculo computacional para lograrlo, y que bueno el niryo one y cualquier robot similar tiene muchas singularidades matemáticas posibles, basta con imaginarse todas formas en las que el brazo puede estar completamente extendido y girarlo para hallar toda una esfera de ellas, estas coinciden con el borde del workspace del robot y no es casualidad, sino una característica de estos diseños de brazo. Así que mejor optamos por buscar singularidades que se encuentren al mover una sola articulación a la vez.

Punto 7.1: Matriz Jacobiana: ejercicios.py -> Opcion 10

Punto 7.2 : Encontrar singularidades.

class_jacobian.py # Programa de prueba, tambien accesible desdeejercicios.py -> Opcion 10 (recomendada)

```
--- Búsqueda de configuraciones singulares ---

Tiempo de cálculo de configuraciones singulares con {t2: 0, t3: 0, t4: 0}: 0.8861 segundos
Tiempo de cálculo de configuraciones singulares con {t1: 0, t3: 0, t4: 0}: 2.6016 segundos

Configuraciones singulares:
Caso 1 (t2=t3=t4=0): [{t1: -1.57079632679490}, {t1: 1.57079632679490}]
Caso 2 (t1=t3=t4=0): [{t2: -1.70541733137745}, {t2: -1.57079632679490}, {t2: 1.43617532221234}, {t2: 1.57079632679490}]
```

Entonces:

Configuraciones singulares:

Caso 1 (t2=t3=t4=0): [{t1: -1.57079632679490}, {t1: 1.57079632679490}]

Caso 2 (t1=t3=t4=0): [{t2: -1.70541733137745}, {t2: -1.57079632679490}, {t2: 1.43617532221234}, {t2: 1.57079632679490}]

Encontramos 2 en rangos theta válidos:

- [0, -1.57079632679490, 0, 0, 0, 0]
- [0, 0, 1.43617532221234, 0, 0, 0]

Los probaremos en el punto 5.

Salida por terminal (última versión, se analizan dos determinantes ya que está preparado para recibir robots con más articulaciones que 6 y aún así encontrar todas sus singularidades, esto lo hace escogiendo columnas diferentes para n determinantes que si no me equivo se calculan según sean necesarios, todo esto con el objetivo de no perder ninguna singularidad de vista aunque en este caso se calculan dos de forma innecesaria. Esto es un pequeño bug que no nos afecta realmente, se repiten soluciones que de todas

```
--- Prueba con todos los valores a pi/2 ---
Sustituyendo: t0: pi/2, t1: pi/2, t2: pi/2, t3: pi/2, t4: pi/2, t5: pi/2)

[[ 0  1  1  0  0  1]
 [ 0  0  0  1  0  0]
 [ 1  0  0  0  1  0]
 [ 0  0  0  0.153 -0.431 0]
 [ 0 0.183 0.183 0 0 0.153]
 [ 0 0 0.210 0 0 0.437]]

Presiona Enter...

--- Búsqueda de configuraciones singulares ---

--- Búsqueda de singularidades con t0 variable ---
Sustituciones: {t1: 0, t2: 0, t3: 0, t4: 0, t5: 0}
Buscando configuraciones singulares para {t0}...
Jacobiana sustituida:
[[ 0  sin(t0)  sin(t0)  cos(t0)  sin(t0)  cos(t0)]
 [ 0 -cos(t0) -cos(t0)  sin(t0) -cos(t0)  sin(t0)]
 [ 1  0  0  0  0  0]
 [ 0 0.183*cos(t0) 0.393*cos(t0) -0.423*sin(t0) 0.423*cos(t0) -0.418*sin(t0)]
 [ 0 0.183*sin(t0) 0.393*sin(t0) 0.423*cos(t0) 0.423*sin(t0) 0.418*cos(t0)]
 [ 0  0  0  0 -0.222  0]]

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (0 encontradas): Ninguna. Tiempo: 0.0705s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluará como solicitado.
Soluciones para Det2 (0 encontradas): Ninguna. Tiempo: 0.0549s

Se encontraron 0 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.1644s
Configuraciones singulares encontradas para t0: []

--- Búsqueda de singularidades con t1 variable ---
Sustituciones: {t0: 0, t2: 0, t3: 0, t4: 0, t5: 0}
Buscando configuraciones singulares para {t1}...
Jacobiana sustituida:
[[ 0  0  0  cos(t1)  0  cos(t1)]
 [ 0 -1  0  0  -1  0]
 [ 1  0  0  sin(t1)  0  sin(t1)]
 [ 0 0.1830000000000000 0.21*cos(t1) + 0.183 0 0.222*sin(t1) + 0.24*cos(t1) + 0.183 0]
 [ 0  0  0  0.183 - 0.183*cos(t1))*cos(t1) - 0.183*sin(t1) 0.183 - 0.183*cos(t1))*cos(t1) - 0.183*sin(t1) 0]
 [ 0  0  0.21*sin(t1) 0 0.24*sin(t1) - 0.222*cos(t1) 0]]

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (2 encontradas): [{t1: -1.57079632679490}, {t1: 1.57079632679490}]. Tiempo: 0.1406s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluará como solicitado.
Soluciones para Det2 (2 encontradas): [{t1: -1.57079632679490}, {t1: 1.57079632679490}]. Tiempo: 0.0180s

Se encontraron 2 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.2015s
Configuraciones singulares encontradas para t1: [{t1: -1.57079632679490}, {t1: 1.57079632679490}]

--- Búsqueda de singularidades con t2 variable ---
Sustituciones: {t0: 0, t1: 0, t3: 0, t4: 0, t5: 0}
Buscando configuraciones singulares para {t2}...
Jacobiana sustituida:
```

formas se filtran más adelante, que estén dentro de los límites y eliminar duplicados básicamente):

```
0 0 0 cos(t2) 0 cos(t2)
0 -1 0 0 -1 0
1 0 0 sin(t2) 0 sin(t2)
0 0.1830000000000000 0.3930000000000000 0 0.222*sin(t2) + 0.03*cos(t2) + 0.393 0
0 0 0 (0.393 - 0.393*cos(t2))*cos(t2) - 0.393*sin(t2) 0 (0.393 - 0.393*cos(t2))*cos(t2) - 0.393*sin(t2)
0 0 0 0.03*sin(t2) - 0.222*cos(t2) 0 0.03*sin(t2) - 0.222*cos(t2) 0

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (4 encontradas): [[t2: -1.70541733137745], [t2: -1.57079632679490], [t2: 1.43617532221234], [t2: 1.57079632679490]]. Tiempo: 0.1800s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluó como solicitado.
Soluciones para Det2 (4 encontradas): [[t2: -1.70541733137745], [t2: -1.57079632679490], [t2: 1.43617532221234], [t2: 1.57079632679490]]. Tiempo: 0.0534s

Se encontraron 4 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.1920s
Configuraciones singulares encontradas para t2: [[t2: -1.70541733137745], [t2: -1.57079632679490], [t2: 1.43617532221234], [t2: 1.57079632679490]]

--- Búsqueda de singularidades con t3 variable ---
Sustituciones: {t0: 0, t1: 0, t2: 0, t4: 0, t5: 0}
Buscando configuraciones singulares para [t3]...
Jacobiana sustituida:
0 0 0 1 0 0 cos(t3) 1
0 -1 0 -1 0 0 -cos(t3) 0
1 0 0 0 0 0 -sin(t3) 0
0 0.1830000000000000 0.3930000000000000 0 0.423*cos(t3) - 0.423*cos(t3) 0.423*cos(t3) 0
0 0 0 0 0.423 0 0.222*sin(t3) 0.423 - 0.006*cos(t3)
0 0 0 0 0 0 -0.222*cos(t3) -0.006*sin(t3)

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (0 encontradas): Ninguna. Tiempo: 0.0276s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluó como solicitado.
Soluciones para Det2 (0 encontradas): Ninguna. Tiempo: 0.0101s

Se encontraron 0 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.0535s
Configuraciones singulares encontradas para t3: []

--- Búsqueda de singularidades con t4 variable ---
Sustituciones: {t0: 0, t1: 0, t2: 0, t3: 0, t5: 0}
Buscando configuraciones singulares para [t4]...
Jacobiana sustituida:
0 0 0 1 0 0 cos(t4) 1
0 -1 0 -1 0 -1 0 0
1 0 0 0 0 0 sin(t4) 0
0 0.1830000000000000 0.3930000000000000 0 0.423 0 0.222*sin(t4) - 0.222*cos(t4)
0 0 0 0.423 0 (-0.423*sin(t4) + 0.222*cos(t4) - 0.222)*sin(t4)
0 0 0 0 -0.222 0 0.222*cos(t4) - 0.222*sin(t4)

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (2 encontradas): [[t4: -3.11675940148949], [t4: -0.0248332521003014]]. Tiempo: 0.1193s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluó como solicitado.
Soluciones para Det2 (2 encontradas): [[t4: -3.11675940148949], [t4: -0.0248332521003014]]. Tiempo: 0.0055s

Se encontraron 2 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.2017s
Configuraciones singulares encontradas para t4: [[t4: -3.11675940148949], [t4: -0.0248332521003014]]

--- Búsqueda de singularidades con t5 variable ---
Sustituciones: {t0: 0, t1: 0, t2: 0, t3: 0, t4: 0}
Buscando configuraciones singulares para [t5]...
Jacobiana sustituida:
0 0 0 1 0 0 1
0 -1 0 -1 0 -1 0
1 0 0 0 0 0 0
0 0.1830000000000000 0.3930000000000000 0 0.423 0 0
0 0 0 0.423 0 0.418 0
0 0 0 0 -0.222 0 0

--- Analizando Determinante 1 (submatriz 6x6 de las primeras 6 columnas) ---
Soluciones para Det1 (0 encontradas): Ninguna. Tiempo: 0.0020s

--- Analizando Determinante 2 (submatriz 6x6 de las primeras 5 columnas y la última) ---
Nota: Para la forma de J_sub (6, 6), Det2 es idéntico a Det1 (columnas: [0, 1, 2, 3, 4, 5]). Se re-evaluó como solicitado.
Soluciones para Det2 (0 encontradas): Ninguna. Tiempo: 0.0010s

Se encontraron 0 configuraciones singulares únicas de las evaluadas.
Tiempo total de procesamiento para find_singular_configurations: 0.0110s
Configuraciones singulares encontradas para t5: []

Configuraciones singulares únicas encontradas:
Singularidad 0: [ 0.00000000, -1.57079633, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 1: [ 0.00000000, 1.57079633, 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 2: [ 0.00000000, 0.00000000, -1.70541733, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 3: [ 0.00000000, 0.00000000, 1.70541733, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 4: [ 0.00000000, 0.00000000, 1.43617532, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 5: [ 0.00000000, 0.00000000, -1.57079633, 0.00000000, 0.00000000, 0.00000000, 0.00000000]
Singularidad 6: [ 0.00000000, 0.00000000, 0.00000000, 0.00000000, -3.11675940, 0.00000000, 0.00000000]
Singularidad 7: [ 0.00000000, 0.00000000, 0.00000000, 0.00000000, -0.02483325, 0.00000000, 0.00000000]

Presione Enter para continuar...
```

```
--- Elipsoides de una configuración aleatoria (validada) ([ 2.3182, -0.6371, 0.4991, 0.1702, 1.3897, 1.7794 ])---
Tiempo de cálculo de la Jacobiana del robot niry_one: 0.1528 segundos
Tiempo de cálculo de la Jacobiana del robot niry_one: 0.1528 segundos

--- Elipsoides de una configuración aleatoria (validada) ([ 2.3182, -0.6371, 0.4991, 0.1702, 1.3897, 1.7794 ])---
Volumen del elipsoide de manipulación y fuerza: 0.001735259866323
Graficando elipsoides... configuración aleatoria

--- Elipsoides configuración nula (0s=0) ---
Volumen del elipsoide de manipulación y fuerza: 0.5450268095105E-8
Graficando elipsoides... configuración 0s=0

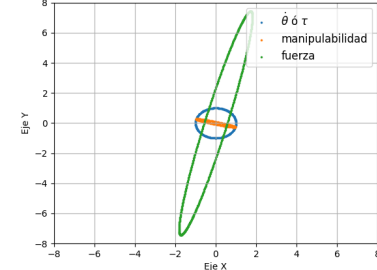
--- Buscando configuraciones válidas (Probaremos a graficar la primera que sea compatible con niry_one) ---
Evaluando 8 configuraciones potencialmente singulares
Probando configuración: [0.0000, -1.5708, 0.0000, 0.0000, 0.0000, 0.0000]
Diccionario de configuración: {t0: 0.0000, t1: -1.5708, t2: 0.0000, t3: 0.0000, t4: 0.0000, t5: 0.0000}
Resultado validación: válida
Configuración válida encontrada: [0.0000, -1.5708, 0.0000, 0.0000, 0.0000, 0.0000]
All joints within limits

--- Usando configuración válida: [ 0.00, -1.57, 0.00, 0.00, 0.00, 0.00 ] ---
Volumen de elipsoides: 2.4540e-40
Graficando elipsoides...
Jacobiana en configuración singular:
0 0 0 0 0 0
0 -1 0 -1 0 -1 0
1 0 0 0 -1 0 -1
0 0.1830000000000000 0.3930000000000000 0 -0.038 0
0 0 0 0.24 0 0.235 0
0 0 -0.21 0 -0.24 0 0

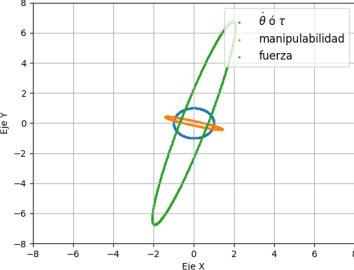
--- Fin de la prueba ---
Presione Enter para continuar...
```

(Graficar algunos elipsoides, por orden de aparición izq -> der)

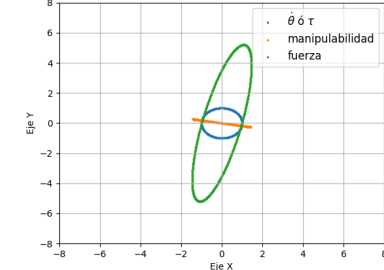
labilidad y Fuerza de [2.31820271 -0.63708018 0.4991365 0.17017083 1.



Elipsoides de Manipulabilidad y Fuerza de (0s=0)



inipulabilidad y Fuerza de Configuración singular [0.00, -1.57, 0.00, 0.00, 0.00, 0.00



Más adelante probaremos lo que sucede cuando el niryo entra en una singularidad, pero antes interpretemos lo que acabamos de ver. El programa acaba de buscar las singularidades que surgen al mover cada articulación individualmente desde la posición cero, y ha graficado 3 configuraciones, una aleatoria, la nula y la primera singularidad que se encontró. Esto se puede hacer para cualquier robot.yaml aunque no sea el Niryo!

Punto 7.3: Comparación de configuraciones y elipsoides de manipulabilidad.

ejercicios.py opción 11: Genera los casos predefinidos y aleatorios, luego es probarlo en el niryo_one.

Comparando configuraciones...

Configuración Cero: [0. 0. 0. 0. 0. 0.] Vol EM: 6.55e-8 Vol EF: 1.53e+7

Configuración Límite Positiva: [2.96673354 0.55323354 1.48373354 2.96673354 1.65773354 2.48673354] Vol EM: 2.82e-7 Vol EF: 3.55e+6

Comparando configuraciones random 8 veces...

Random Config 1: [2.9 -0.46 1.33 2.73 -0.36 1.78] Vol EM: 1.18e-7 Vol EF: 8.48e+6

Random Config 2: [2.56 -0.83 1.13 -2.71 1.12 1.49] Vol EM: 2.17e-5 Vol EF: 4.61e+4

Random Config 3: [0.09 -1.07 -0.18 1.51 0.76 0.81] Vol EM: 8.86e-5 Vol EF: 1.13e+4

Random Config 4: [1.76 -0.98 -0.02 -0.65 -0.96 2.33] Vol EM: 1.42e-4 Vol EF: 7.03e+3

Random Config 5: [1.61 -1.45 0.77 -1.23 -0.45 0.06] Vol EM: 2.21e-5 Vol EF: 4.52e+4

Random Config 6: [-1.05 -1. -0.62 -2.04 -0.75 0.71] Vol EM: 2.86e-5 Vol EF: 3.50e+4

Random Config 7: [2.14 -0.94 0.99 -1.75 0.09 -0.77] Vol EM: 9.16e-7 Vol EF: 1.09e+6

Random Config 8: [-1.16 0.37 -0.3 -1.88 1.43 0.06] Vol EM: 4.52e-5 Vol EF: 2.21e+4

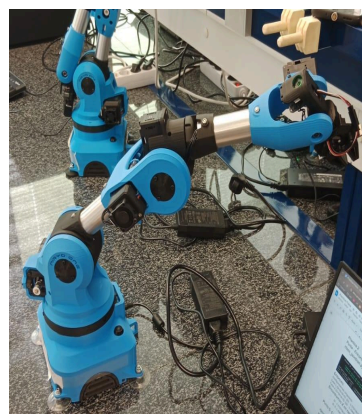
Nota. Las configuraciones singulares tienen EM de 10e-18. Cosa que las anteriores cumplen con creces hasta se encontraron del orden de 10e-40! Eso sí es una singularidad.



Config 0



Config 1



Config 2



Singularidad

```
0. Salir.

Seleccione una opción: 11
  Robot 'niryo one' creado.
  Tiempo de ejecución de get ejes helicoidales: 0.0005 segundos
  Tiempo de cálculo de la Jacobiana del robot niryo_one: 0.0694 segundos
Comparando configuraciones...
Configuración Cero: [0. 0. 0. 0. 0. 0.] Vol EM: 6.55e-8 Vol EF: 1.53e+7
Configuración Límite Positiva: [2.96673354 0.55323354 1.48373354 2.96673354 1.65773354 2.48673354] Vol EM: 2.82e-7 Vol EF: 3.55e+6

Comparando configuraciones random 8 veces...
Random Config 1: [2.9 -0.46 1.33 2.73 -0.36 1.78] Vol EM: 1.18e-7 Vol EF: 8.48e+6
Random Config 2: [2.56 -0.83 1.13 -2.71 1.12 1.49] Vol EM: 2.17e-5 Vol EF: 4.61e+4
Random Config 3: [0.09 -1.07 -0.18 1.51 0.76 0.81] Vol EM: 8.86e-5 Vol EF: 1.13e+4
Random Config 4: [1.76 -0.98 -0.02 -0.65 -0.96 2.33] Vol EM: 1.42e-4 Vol EF: 7.03e+3
Random Config 5: [1.61 -1.45 0.77 -1.23 -0.45 0.06] Vol EM: 2.21e-5 Vol EF: 4.52e+4
Random Config 6: [-1.05 -1. -0.62 -2.04 -0.75 0.71] Vol EM: 2.86e-5 Vol EF: 3.50e+4
Random Config 7: [2.14 -0.94 0.99 -1.75 0.09 -0.77] Vol EM: 9.16e-7 Vol EF: 1.09e+6
Random Config 8: [-1.16 0.37 -0.3 -1.88 1.43 0.06] Vol EM: 4.52e-5 Vol EF: 2.21e+4
Presione Enter para continuar...
```

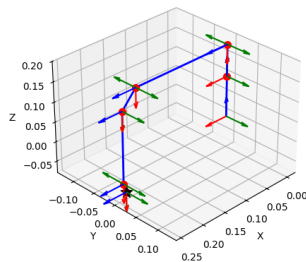
Nota. Actualización sobre el volumen (esto una medida de proporción realmente) es el mismo para el EM como para el EF (el valor en Vol EM es el bueno, se hizo de la manera que se porque en la propia práctica así se indicaba de forma errónea según el profesor). El código posterior a esta captura está corregido.

Punto 7.4. Cinemática Inversa.

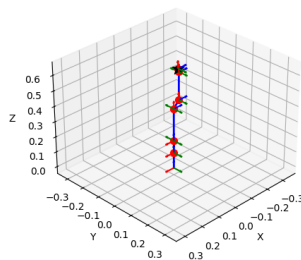
Este último punto lo completé fuera del laboratorio, he tratado de validarlo por mi cuenta y este ha sido el resultado. La explicación del cómo funciona y el método usado (una versión de Newton-Raphson) está dentro del código, así como un programa extra para graficar y animar el robot a modo de simulación:

- [problema_cinematico_inverso_gen.py](#)
- [class_robot_plotter.py](#)
- [class_robot_plotter_prueba.py](#)
- [Carpeta con Visualizaciones_niryo_one](#)
- Ejemplo del contenido:

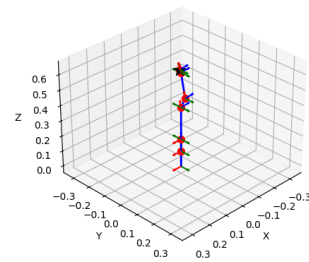
Configuración singular 1 [0.00 , -1.57 , 0.00 , 0.00 , 0.00 , 0.00]



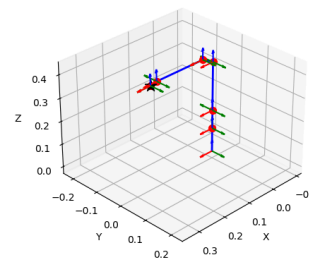
Configuración singular 3 [0.00 , 0.00 , 1.57 , 0.00 , 0.00 , 0.00]



Configuración singular 2 [0.00 , 0.00 , 1.44 , 0.00 , 0.00 , 0.00]



Configuración singular 4 [0.00 , 0.00 , 0.00 , 0.00 , -0.02 , 0.00]



El programa de cinemática inversa se adecua a lo pedido en el primer punto, limitando las soluciones según los límites de las articulaciones para asegurarnos de que sea alcanzable por el robot. Decir que por defecto toma de semilla la configuración nula, y que como cualquier semilla esta puede no ser adecuada para ciertos puntos/orientaciones del efecto final, que por defecto [roll, pitch, yaw] (orientación) y [x, y, z] (posición), será [0,0,0] [0.1, 0.1, 0.1]. La mejor forma de entenderlo es ejecutar el programa y observar cómo en cada iteración nos acercamos a la configuración deseada siempre que el las velocidades en el vector giro convergan a 0. La implementación resultante es muy potente, donde en unas pocas iteraciones encontramos la configuración deseada, dado que tratar que el error en las velocidades y matriz de transformación homogénea del efector final sean 0 no es viable damos por defecto un margen de error de $10e-10$, señalar que márgenes menores como $10e-20$ o $10e-40$ son viables pero el resultado final no resulta mucho más preciso que $10e-10$ que resulta más que suficiente y equilibrado.

Con este programa podemos describir cualquier trayectoria que queramos y aplicar cinemática inversa para conseguir un array de configuraciones que nos sirva para describir el movimiento sobre dicha trayectoria del robot, por ejemplo una trayectoria circular: Ver [trayectoria_circular_niryo.gif](#) (Es posible que el salto inicial sea por acercarse a una singularidad).